A Meta-Analysis of Deep Learning Models for Vehicles-Pedestrian Detection with Framework Proof of Concept

Harith Bin Mohd Izani
Department of Computer Science
Kulliyyah of ICT, International Islamic University Malaysia
Kuala Lumpur, Malaysia
harithizani@gmail.com

Muhammad Hariz Bin Hasnan
Department of Computer Science
Kulliyyah of ICT, International Islamic University Malaysia
Kuala Lumpur, Malaysia
harizhasnan32@gmail.com

Amelia Ritahan Ismail
Department of Computer Science
Kulliyyah of ICT, International Islamic University Malaysia
Kuala Lumpur, Malaysia
amelia@iium.edu.my

***Abstract*—The classification and localization of objects (object detection) have gone through many iterations and refinement over the course of its history. The issue of knowing which algorithms is most suited for which applications are not well explored especially within the traffic engineering domain. We review up-to-date literatures to determine which has the best overall potential and test models with the best prospects and compile the findings of both the meta-analysis and the results gained from our proposed framework design as proof of concept.**

Keywords—Object Detection, Deep Learning, Convolutional Neural Network (CNN), Single Shot Detectors (SSD), Two Stage Detectors, Internet of Things (IOT), Average Precision (AP), Total Loss (TL), Luminance, Web Content Accessibility Guidelines (WCAG), Berkeley Deep Drive (BDD)

I. Introduction

A. Project Overview

A road and street are full of objects ranging from vehicles of multiple sizes all the way to billboards. A traffic engineer and city planner could look at an image of a street/road and easily identify cars, pedestrians etc., however if they want information (data) such as "how many cars go through this road each day?" or "how busy is the pedestrian traffic on this street at any given time?" they would find it hard pressed to count individual objects to obtain that information-in this case data-to make well informed decisions. The work of identifying these entities could be offloaded to a computer, these computers would use image detection algorithms in order to distinguish between each entity on the road/street, thus automating the process of gathering data. The problem however is knowing which algorithm to use and under what conditions, for example, which algorithm is suitable for day time and which for night, all this has to be able to perform on a computer with very limited resources and computing power such as internet of things (IOT) devices. This research is conducted to test the performance of single-shot object detection model MobilenetV2 in detecting vehicles and pedestrian on roads and streets. Two categories will be used to evaluate effectiveness: (a) light; how bright is the environment of the image and (dark): environments where it is difficult to see objects. The dataset "Berkely Deep Drive Dataset" will be used as an analog to real world environments as we think that this dataset is the most difficult to train an object detection model on due to how the data is gathered. The result of the meta-analysis should give insight on how to develop a framework that could perform well in all categories, meaning a combination of objects detection models that can detect pedestrians and vehicles under both category (a) and (b). This framework could contribute by creating a robust system for traffic engineers and city planners to use for gathering road/street data reliably.

B. Problem Statement

Most datasets pertaining to the domain of this research are "ideal", meaning the images are almost always clear (very high resolution with almost every object visible), the objects are of reasonable distance (pedestrians and vehicles are not too far or too close) and are usually set during day time, giving ample lighting to objects. There is a lack of meta-analysis on object detection algorithms on non-ideal datasets(datasets which are not "ideal" as previously mentioned), or if the dataset does have night and day environments, the literature doesn't emphasize on the model's performance for those categories, merely mentioning its overall accuracy, which indicate that these algorithms are not reflective of their performance in the real world, another aspect is there are almost no meta-analysis that touch on lightweight models for this use-case.

C. Objective

1. To determine if producing a framework that can perform well for categories: (a) light, (b) dark is possible
2. To create a meta-analysis on major up-to-date deep learning-based object detection models. Specifically, EfficientDet and MobilenetV2.

3. Analyze and compare performance of each model tested.

4. To determine if a portable traffic data gathering tool using the proposed framework is possible.

D. Research Scope

The research will only focus on road/street vehicles and pedestrians. This project aims to only count the most important aspect of what constitutes "traffic", thus road signs and traffic cones will not significantly help in that regard and will require the computer to waste resources on detecting something not very significant.

E. Research Significance

1. The framework(if viable) will contribute in major areas regarding traffic such as, self-driving technology, road monitoring systems, traffic engineering etc. It will contribute in the development of traffic engineering domain.

2. This project should steer other projects to use suitable algorithms for their objects and applications, especially IOT applications.

II. Methodology

A. Research Design

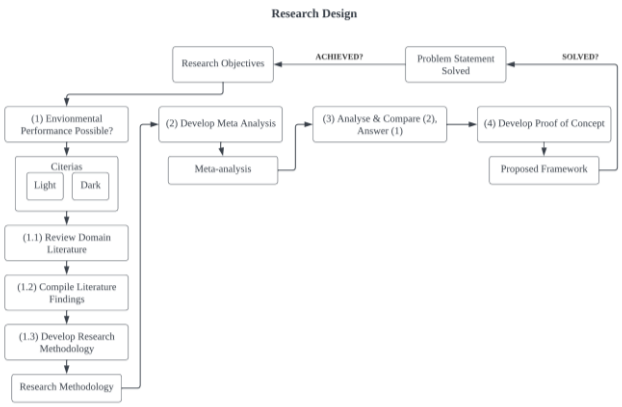


Fig. 1: Research Design

This research has 3 stages, (1): Literature Review, (2): Meta-analysis and (3): Result Analysis. The 3 stages are from the objectives of this paper and the stages are performed in sequence with the last stage answering the problem statement.

B. Stage 1: Literature Review

From the objectives, the models must perform well in 2 criteria introduced in the project overview above. (a):light and (b):dark. From these two criteria stage 1.1 aims to

1. Justify this paper's problem statement by reviewing journals that do not focus on dataset composition (whether the dataset contains day or night) and journals that do utilize dataset containing criteria (a) and (b) however do not emphasize or specifically test performance of models on (a) and (b).

2. Review domain literature to understand trends in object detection and determine models suitable to be tested to satisfy this paper's problem statement, emphasizing IOT or lightweight use-case.

3. Review domain literature to determine techniques that can differentiate model performance on criteria (a) and (b). e.g., gray scaling images to simulate model training on dark environments.

C. Stage 2: Meta-Analysis

From stage 1.2, the research methodology is followed to develop the meta-analysis. The meta-analysis is the compilation of performance results of models chosen from stage 1 tested on criteria (a) and (b). The metrics include average precision(AP), total-loss(TL), bounding box tallies, steps-per-second and training graphs.

D. Stage 3: Result-Analysis

Analysing results from the meta-analysis should give definite results in which object detection models and techniques

1. Perform best for criteria (a) and (b)

2. Can be arranged into a framework to satisfy problem statement (criteria (a) and (b) as well as computational resource and training time)

E. Stage 4: Proof of Concept

Stage 3 should definitely state which model is best suited for criteria (a) and (b). If only one model is the best performer in both criteria, then the framework would only consist of one model, however the input would be augmented (in this case gray scaled) so that it would perform optimally under both environmental criteria (a) and (b). If two models; each performing optimally in one of the criteria. Then the resulting framework would consist of both models, the input will not be augmented as the models should be able to hyper-specialize in detecting objects for each criterion.

The reason for not combining both input augmentation as well as both models is due to the computational resources needed. A relatively weak microprocessor will be used such as the one found on the Raspberry Pi 4B. This is for the purpose of portability in order to satisfy stated objective 4.

III. Literature Review

A. Object Detection

In the past two decades, object detection has seen continuous development and research within the community. Since then, two approaches in the field of object detection have emerged: traditional object detection and Deep Learning-based object detection.

1) Traditional Object Detection

In early studies of object detection, the traditional approach was the only feasible method to implement and classify objects. This approach has three stages: (1) formative region selection, (2) feature extraction, and (3) classification [2]. Formative region selection determines the position and presence of the object by scanning through the whole image multiple-scale sliding window [4]. Features extraction refers to the technique used to obtain handcrafted features [3] that contain representative information about the object that can be processed. Classification is the technique used by the detector to classify the object based on the features extracted. This approach is the basis of traditional object detection and is the reason it became obsolete after the emergence of deep learning-based object detection. In the first stage, scanning through the whole image is computationally exhaustive, and

produces many redundant windows depending on the position, size, and amount of object presence [4]. Hand-crafted features are extracted features from images according to a manually predefined algorithm based on an expert's knowledge [3], which is not always sufficient in describing all types of objects.

2) Deep Learning-Based Object Detection

In recent decades, deep learning-based models have dominated the interest of researchers in the field of object detection. This method of learning is based on an artificial neural network, which aims to simulate the human brain system [4]. This approach requires huge amounts of data to generate the features and deeper architecture compared to traditional models [2]. Deep learning has existed since the late 1900s, but due to overfitting, lack of large-scale datasets, limiting computation power, and performance issues compared with other tools at the time, it slowly faded. However, in the 2000s, with the rise of large-scale datasets, advancement of highly parallel computing systems (GPU), and improvement in the design of network structures and training strategies, deep learning implementation began to show its performance with full capacity [4]. This approach utilizes CNN for visual feature extraction [2], which extracts what is known as a learned feature. Learned features perform equally well with handcraft features on small-scale homogeneous datasets, but significantly outperform the latter on heterogeneous data, and feature generalization on unseen data [3]. This is crucial for object detection as the images will come in diverse appearances, illumination conditions, and backgrounds [4].

The current approach has two methods of detection, One-stage object detectors, and two-stage object detectors. One-stage object detectors are designed to identify and locate objects in a single stage, without the need for a separate region proposal step. These methods typically use a single neural network to directly predict the class and location of objects in the image. One-stage object detectors tend to be faster and more efficient than two-stage object detectors but not as accurate. Two-stage object detectors, on the other hand, use a two-step process to identify and locate objects. In the first stage, a region proposal network (RPN) is used to generate a set of candidate object regions, or proposals, in the image. In the second stage, a separate classifier is used to classify the objects and refine the location of each object within the proposals. Two-stage object detectors tend to be more accurate than one-stage object detectors, but they may be slower and more computationally intensive [2][10].

In recent years, a new branch of object detection has recently garnered attention called lightweight object detection. Zaidi et al paper [1] provide a comprehensive survey of information in the field of object detection and have noted the emergence of lightweight models aimed to design a small and efficient network for resource-constrained applications such as IoT and edge devices. The authors discuss several prominent networks for lightweight models, which are mainly used for single-stage detectors. They also performed a performance comparison between several models with varying architectures. Our experiment aims to compare the performance of a single-stage model and a lightweight model from the study [1], and have chosen MobileNetv2[11] and EfficientDet [7].

B. Potential Models

In this work, SSD Mobilenetv2 FPNLite 320x320, SSD mobilenetv2 FPNLite 640x640, EfficientDet D0, and EfficientDet D2 are the models use for the experiment, obtained from TensorFlow Detection Model Zoo.

1) SSD MobilenetV2 FPNLite

This model is a combination of (1) MobileNetV2[11] network for feature extraction, (2) Single Shot Detector (SSD) [12] for classification, and (3) Feature Pyramid Network Lite (FPN Lite) [13] to provides more contextual information for the feature extraction. MobileNetV2 [11] architecture was designed for efficient computation on mobile and embedded devices, using a combination of depthwise convolution and point-wise convolution to reduce the parameters and computational cost. SSD [12] is an object detection algorithm that utilizes a set of default bounding boxes for predicting the location and object classes within an input image. The original MobileNetV2 SSD [15] architecture as shown in figure 2 lacks the ability to detect small objects, therefore, FPN Lite [13] is incorporated to improve this issue. The pixel number at the end of the model, SSD Mobilenetv2 FPNLite 320x320 and SSD mobilenetv2 FPNLite 640x640 indicate that the model automatically scales the input image into the corresponding pixel size.

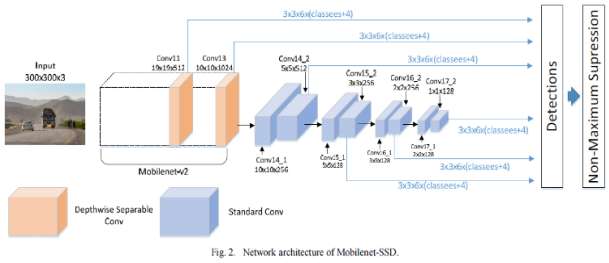
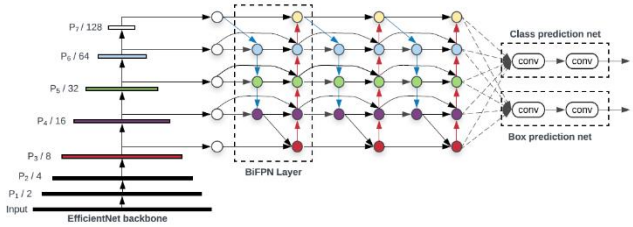


Fig. 2: MobilenetV2-SSD Architecture

2) EfficientDet

EfficientDet [7] is a family of object detectors designed to be accurate, efficient, and scalable. The model's architecture shown in figure 3 consists of (1) EfficientNet network for feature extraction, (2) bi-directional-feature network (BiFPN) for easy and fast multi-scale feature fusion, connected to (3) class and box prediction net for classifying the class and bounding box of the object. EfficientNet architecture uses a novel scaling method designed for efficiency in size, computational cost, and number of parameters. BiFPN is a feature pyramid network that uses bidirectional top-down and bottom-up architecture allowing high-level and low-level details to be extracted. EfficientDet D0 and D1 are different scaling versions of the family.

Fig. 3: EfficientDet Architecture



C. *Current Literatures*

Dataset plays an important role in object detection. It shaped how the algorithm perceived a given image based on the dataset it is trained upon. For road object detection, one issue in a dataset is the lack of time zone available. Time zone is typically divided into two periods: daytime and night-time, which in a dataset, would produce vastly different images. The daytime images are considered "ideal" as the images are much clearer than their night-time counterparts. However, both images are crucial to creating a robust real-time detector that can detect objects at any time of the day. The current problem in literature [8][9] is that only the daytime images are utilized or daytime and night-time images are utilized but fail to address the disparity between them, which both cases are not suitable for road detection. Kim et al [8] conducted a comparison study on multiple deep-learning object detection models to determine which model is the most optimized for vehicle type recognition in real-time. Chen et al [9] produced an object detection, localization, and tracking system for smart mobility environments through extensive research and comparison of models and methods on Cityscape[16] and KITTI[17] datasets. Figure 4 shows a snapshot of multiple images from the Cityscapes dataset, consisting of only "ideal" or daytime images.

Fig. 4: A Snapshot of Multiple Images from the CityScapes Dataset

Haris and Glowacs[2] conducted a study comparing deep learning-based algorithms for road object detection. They found that current literature on this topic has several limitations, such as using low-fine-grained recognition datasets, focusing on only one class of objects (usually vehicles), and lacking in-depth analysis of different models. To address these issues, the authors used modern, high-quality datasets, BDD100k[18], and state-of-the-art models from single and two-stage detectors to thoroughly test the performance of various object detection models under different road conditions and scenarios. They evaluated the model's precision and recall with and without occlusion and truncation. The study also looked at the processing time of the GPU and CPU individually, and the generalization ability of the models by various weather scenarios, through different times of the day. From this analysis, they found that the two-stage models generally performed better in terms of accuracy, while the single-stage models were faster. Overall, the results of the study provide a comprehensive comparison of the different object detection models and their performance. Figure 5 shows a snapshot of multiple images from the BDDK100 dataset, consisting of both daytime and night-time images.

Fig. 5: A Snapshot of Multiple Images of the BDDK100 Dataset. Day-Time Images[Top] and Night-Time Images[Bottom]

The literature [2] addresses the disparity in performance between night and day but did not mention any method to improve them. One method that may solve this issue by Gray scaling the dataset. Bui et al [5] conducted an experiment comparing the performance of RGB images and turned grayscale images on object recognition using their own and other classifiers. The experiment concluded that object recognition based on grayscale images outperforms RGB images in terms of accuracy and computation efficiency. The authors noted that computing features for extra-class classification are most suited for grayscale images. Road Images fall under extra-class images as each class has a unique shape and texture. Figure 6 shows a snapshot of daytime and night-time image from the DDK100 dataset, before and after Gray scaling.

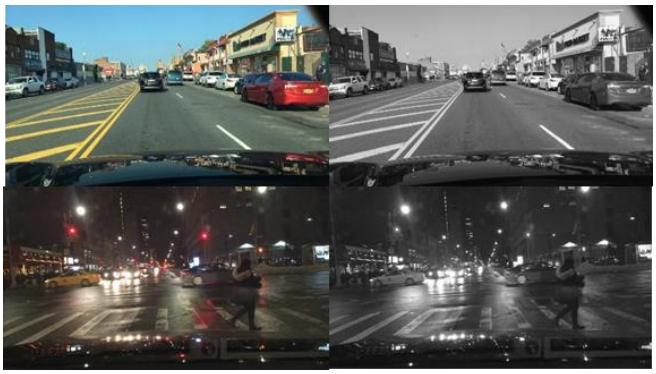


Fig. 6: A Snapshot of Images from Berkely DeepDrive Dataset. Before Gray Scaling[Left] and After Gray Scaling[Right]

IV. META-ANALYSIS

A. Research Methodology

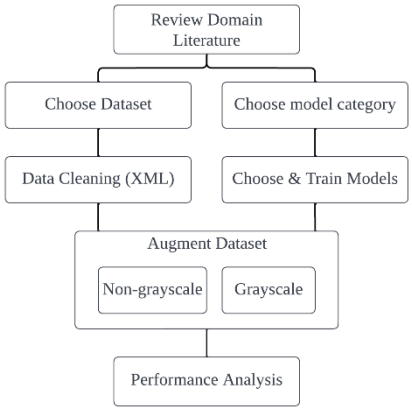


Fig. 7: Research Methodology

The machine specification used for this research are as follows:

1. Processor: AMD Ryzen 5 5600H 3.3 GHz
2. RAM: 16 GB
3. GPU: Nvidia GTX 1650 4 GB VRAM
4. Operating System: Windows 11

The first step of our research is to perform literature review to determine potential models and appropriate techniques. The dataset most suitable for this research is the Berkeley Deep Drive Dataset due to its difficulty. We think it is an appropriate analogue to the real world, the logic is that if the models trained and tested on this dataset are high performing, then it should consequently be a good candidate to meet objective 1. Many samples from the dataset are so difficult that even humans would have a hard time guessing some of the objects as seen below.

Fig. 8: Picture 25 Cars Blending into Background

There are a total of 13 objects annotated by the Berkeley Deep Drive team, the objects are

1. Bicycle
2. Bus
3. Car
4. Motorcycle
5. Pedestrian
6. Pedestrian
7. Rider
8. Train
9. Truck
10. Other Vehicle
11. Trailer
12. Street Sign
13. Street Light

The json format is hard to read and modify, so we decided to change the label files into XML. Using the “xml” python package we convert the XML structure into a tree that is modifiable, with this we remove any bounding box coordinates that are outside the scope of this research, meaning removing “Street Sign” and “Street Light”. The BDD dataset contains 69,862 training images and 1000 testing images or all environments and conditions, meaning it contains both criteria (a)light and (b)dark.

From the literature review section, the most viable object detection model categories are single-shot detectors and lightweight. We chose these 2 categories because they are the most likely candidates that will be able to run on a machine with limited computing power and memory. Another factor that has to be met is “accessibility”, the idea is for other researchers to be able to replicate this work’s results and findings. We decided to go with models that are available on tensorflow model zoo published to the public by Tombstone, a-googler, vighneshbirodkar, srjoglekar246, pkulzc, khanhlvg on a github repository. The models published are trained on COCO dataset, however the dataset is much simpler than the BDD dataset used for this research, thus for the models to be usable, we use transfer learning to retrain the models on the BDD dataset. For single-shot detectors EfficientDet D0 and EfficientDet D2 are chosen due to its intended purpose of scalability(refer to literature review). EfficientDet D0 is to test baseline performance of the EfficientDet model and EfficientDet D2 will be used to test the model with a higher resolution(up to the limit of our research machine). For lightweight models made to specifically run on IOT devices, mobilenet is the most suitable due to its history of being used to run on mobile devices etc. The same applies to mobilenet, mobilenet 320 will be used to gauge baseline performance and mobilenet 640 will be used to see performance on higher resolutions up to the limits of research machine. The model training will require a lot of computing power however when exported and run on small devices, the computing power of the research machine is no longer needed as the models are no longer updating any weights or parameters.

To fulfil the project objective of performing well in (a) and (b), the models must be able to detect objects within any conditions. From the BDD dataset all of the conditions all fall within 2 categories of (a) and (b), for example if it is day time however in the picture it is raining, the end result is the picture becomes (b)dark. The metrics of determining which is (a) and (b) are as follows

1. (a) Light
    a. The object must be very visible (meaning it is easily detectable by the human eye).

b. The object must be distinguished in terms of color (the colors are easily separated by the human eye).

c. The overall hue of the picture is very bright with little to no white noise or distortions when the image was captured.

d. Example of a (a) Light image

Fig. 9: Example of (a) Light Image

2. (b) Dark

a. The objects are not very visible (meaning it is difficult for humans to detect the object)

b. The object colors are almost blending in with its background.

c. The overall hue of the picture is dark/darker with information loss, white noise or distortions when the image was captured.

d. Example of a (b) Dark image

Fig. 10: Example of (b) Dark Image

To satisfy object 1 of this research, the models have to be trained on both (a) and (b) conditions, thus the BDD dataset could be separated into category (a) and (b). To separate the dataset, we have determined that there are 3 potential methods.

1. Separate using RGB values
2. Separate using brightness values (light intensity/lumens)
3. Separate using gray scaling

The first method involves finding the average RGB value of an image and then setting a threshold and determining whether it belongs in category (a) or (b). The problem with this method is if the image has many objects or backgrounds that are of a darker colour(like a large dark coloured building), the image could be mis-categorized as (b) even though the image is (a).

The second method uses openCV "HSV colour model" which determines from a scale of 0-100 an aggregation of hue, saturation and value. The closer the value is to 100, the brighter the image. The shortcoming of this technique is if the image has a shaded area, for example a metro way in New York, the image will be classified as (b). It is essentially the same problem as the first method however it is using HSV value rather than just RGB.

The last method is briefly discussed in literature review of this research, it involves changing all RGB values of an image into its equivalent grey counterpart. This same method is used when performing edge detection, this property can be exploited to help the models learn object features for (b)Dark images. For an image like the one below

Fig. 11: Example Blending of Colors

The colors would start to blend and a human would have trouble distinguishing objects, however if grayscale the gray equivalents would make the colors value(meaning the brightness on the grayscale's scale) far apart and easing the process of distinguishing an object from its foreground. For example if rgb value of an object is 0,0,0 and foreground is 50,50,50

Fig. 12: RGB 0,0,0 and 30,30,30 respectively

Gray Scale Formula

$$Y = 0.25R + 1G + 0.5B$$

When grayscale, the colours turn into the colours below, it may look insignificant, however if inspected using a high resolution version of this document, the colour in Figure 10, the colour on the right has a very distinct green tint, green is the most sensitive colour to the human eye[20], thus increasing it will not only help distinguish the object and the foreground better for humans, but better for machines as well, due to the difference in Gray colour scale.

Fig. 13: Gray Scale Version of Figure 12

Using the formula, a picture can turn into a very bright and very distinguishable version of itself as seen down below in Figure 11. With this, we decided to choose grayscale as an analogue or replacement to (b)dark data to train the model for 2 reasons

1. It is excellent for edge detection, thus the machine should learn features of an object more accurately which is needed for (b)Dark scenarios, however we expect the recall to be lower due to the model learning very specific features of an object and if it doesn't fit it even slightly it will fail to detect.
2. It is a more reliable substitute as (b)Dark data as opposed to the previous 2 methods where the chances of mis categorization was high and thus could lead to the model overfitting.

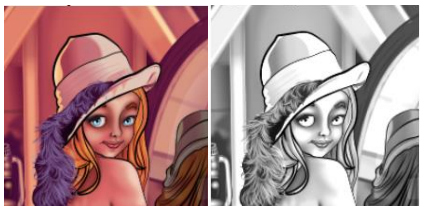


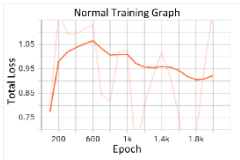
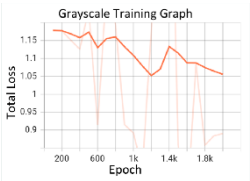
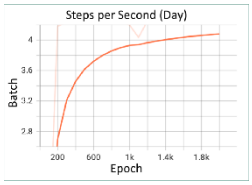
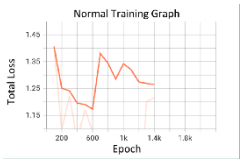
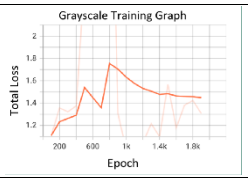
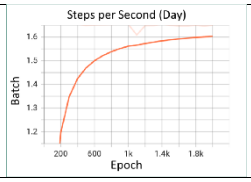
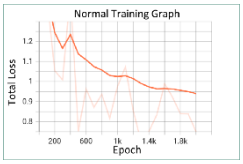
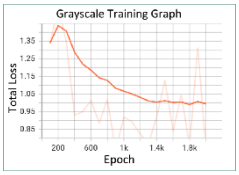
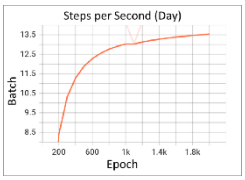
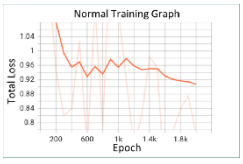
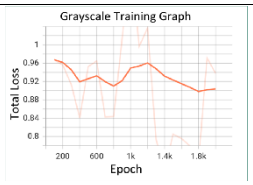
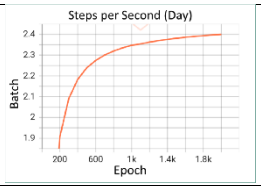
Fig. 14: Gray Scaling Distinguishing Example

There is a higher degree of separation in the figure above especially on the right side of the image, to a human eye the objects are "popping" and are more separated to its background compared to the left image.

B. *Testing & Experimentation*

The training and testing results of EfficientDet D0 512x512, EfficientDet D2 768x768, Mobilenet v2 FPNlite 320x320, Mobilenet v2 FPNlite 640x640 on both grayscale and non-grayscale versions of the BDD dataset are compiled in a table below. The testing done below has batch size of 1 image and an epoch size of 5000, meaning the total training done on the images is

TABLE I. TRAINING RESULTS

Model	**Results**			
	Average Precision	***Normal Training Graph***	***Gray Scale Training Graph***	***Training Speed***
EfficientDet D0 512	1.4^{-4}			
EfficientDet D2 768	0.4^{1}			
Mobilenet v2 320	0.01^{1}			
Mobilenet v2 640	0.025^{1}			

C. Training Graph

To determine which model is most suitable, we look into which model has the most "potential", meaning which model, if trained on a much larger epoch or trained indefinitely, has the highest possibility of reaching 90% AP. To determine this a somewhat gradual downward slope of the training graph is desired(both normal and grayscale training). Table 1 indicates that Mobilenet v2 320 and Mobilenet v2 640 have the best training graph shape. Efficient D0 512 does not have a downward slope and instead has an upward slope while the grayscale graph has a sharp spike in the middle, we think this is due to a local minima problem, meaning the model has learnt the features of an object however since the dataset is large, a completely different vehicle with vastly different features is introduced and it leads to retraining, it is an indication of poor generalization. EfficientDet D2 768 has the worst graph, the same local minima problem is present however it is very pronounced. In theory EfficientDet D2 768 should have the best overall metrics, it has the deepest convolutional network and the highest operating resolution amongst the 4 models. EfficientDet D2 768 has the highest AP, however the graph shows otherwise, we think that this model has the highest precision however has very low recall, meaning it is guessing correctly and precisely however it is not guessing very many times. This would later be confirmed on the next evaluation step below.

D. Performance on Testing Images

The next step is to look into its actual performance on images it has never seen before to test generalization and consequently its potential performance in the real world. The detection are categorized into "quality" ranging from 1-5, the specifics are as follows (a detection only counts if any of the requirements are met, if no prediction is made, it will not be counted within the tally).

TABLE II. DETECTION QUALITY TABLE

Quality	*Requirements of Detection*
1	The prediction bounding box does not accurately encompass any relevant objects
2	Most prediction bounding boxes are of category 1, however there is at least 1 bounding box that at least wraps a relevant object although the precision isn't very good.
3	More than half of relevant objects have bounding box of acceptable precision however the prediction still has many that is of category 2
4	Most of the objects have bounding box(not all), with a few mistakes of category 3
5	All objects have a bounding box of acceptable precision, if the previous condition is met, a few mistakes will not be taken into account.

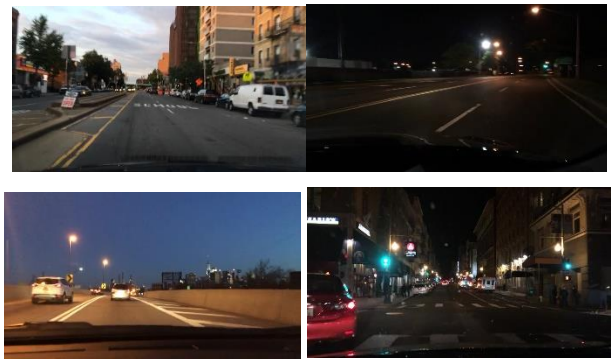


Fig. 15: Testing Environment Substitutes

The performance on (a)light and (b)dark images that are manually separated will be tallied and compiled in the tables below as well as some of the images encompassing most real life equivalent environments(recall all environments fall within category (a)light and (b)dark, however for testing, we separated into day and night because it is the most relevant equivalent to test (a) and (b)). The testing images are not grayscale or doctored in any shape or form, the testing images are supposed to mirror real settings, the techniques discussed above are only substitutes for training.

TABLE III. EFFICIENTDET D0 512X512 TESTING RESULTS

Training Data & Environment	Results	
	No. correct predictions	*Average prediction rating*
Normal – Day Time	1	2
Normal – Night Time	1	1
GrayScale – Day Time	0	0
GrayScale – Night Time	0	0

TABLE IV. EFFICIENTDET D2 768X768 TESTING RESULTS

Training Data & Environment	Results	
	No. correct predictions	*Average prediction rating*
Normal – Day Time	0	0
Normal – Night Time	0	0
GrayScale – Day Time	0	0
GrayScale – Night Time	0	0

TABLE V. MOBILENET V2 FPNLITE 320X320 TESTING RESULTS

Training Data & Environment	Results	
	No. correct predictions	*Average prediction rating*
Normal – Day Time	5	3.6
Normal – Night Time	0	0
GrayScale – Day Time	5	2.6
GrayScale – Night Time	1	2

TABLE VI. MOBILENET V2 FPNLITE 640X640 TESTING RESULTS

Training Data & Environment	Results	
	No. correct predictions	*Average prediction rating*
Normal – Day Time	8	3.5
Normal – Night Time	2	2
GrayScale – Day Time	7	4
GrayScale – Night Time	1	5

"Average Prediction Rating" is used instead of a confusion matrix, this is due to confusion matrix only telling us how many predictions a model makes and how frequent it predicts, it doesn't give any details on the "quality" of those

predictions. The use of Prediction Rating gives a more complete picture as to how well the model actually performs.

$$Average\ Prediction\ Rating = \frac{\sum(Prediction\ Detection\ Quality)}{N(Predictions)}$$

For example if model A makes 3 predictions of quality 3,4,5 the average prediction rating would be.

$$Average\ Prediction\ Rating = \frac{3+4+5}{3} = 3$$

Taking a look at the tables above, the model that has the best overall performance is Mobilenet v2 FPNlite 640x640 for both day and night time environments(testing substitute for (a) and (b)). The model not only makes more predictions than other models, it also has higher prediction quality(as seen in its Average Prediction Rating).

This model when trained on the normal dataset performs the best in day time environments and consequently category (a)light. For category (a) it has the highest number of predictions with the second highest daytime prediction quality of 3.5. The most important aspect of category (a) is recall, most models are already able to predict this environment, thus having the most recall will give the traffic engineers the best estimates of the daytime traffic conditions as opposed to having very low recall yet with very high precision as is the case with EfficientDet D2 previously discussed in the graph discussion above.

The most important aspect of night time environments and consequently category (b) is high precision as opposed to high recall like that of day time/category (a) predictions. This is due to most models having a difficult time detecting objects, thus a model with higher precision with acceptable recall will provide traffic engineers with better quality statistics. For this case when trained on grayscale version of dataset, Mobilenet v2 FPNlite 640x640 has the best night time performance with the highest average prediction rating of 5.

E. Result Analysis & Proposed Framework

To satisfy this research objective, the model has to perform well under all conditions, all conditions eventually distil into 2 main categories (a) and (b), from the meta-analysis above, the model that performs the best for both (a) and (b) is the Mobilenet v2 FPV lite 640x640, theoretically a framework consisting of 2 of these models, one trained on normal dataset for its high recall and acceptable precision on category (a) performance and the other trained on grayscale dataset for its high precision and acceptable recall. A potential framework that can account for both environments of category (a) and (b) could be similar to the framework below

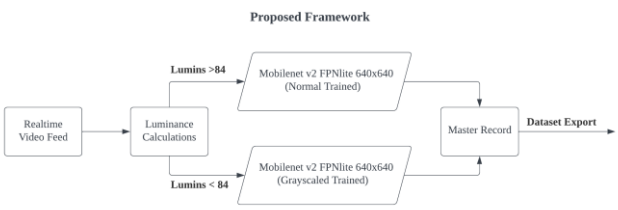


Fig. 16: Proposed Framework

An input device would feed real-time or input an image/video into a luminance calculator function. There are many methods to determine "brightness" of an image/video. The method used for this project is the "brightness" or luminance value set by WCAG. The measurement is the relative "brightness" perceived by the human eye. The formula is as follows.

$$avg\ RGB = \frac{\sum(RGB)}{No\ of\ Pixels}$$

$$lumen\ (whole\ frame) = (0.2126 * R) + (0.7152 * G) + (0.0722 * B)$$

To determine which lumen value falls under criteria (a) and (b), 10 images were chosen at random from classification 1) Bright 2) Dim and 3) Dark each for 30 images total. The conditions for which falls under classification is determined by the table below.

TABLE VII. IMAGE CLASSIFICATION CONDITIONS

Classification	***Are objects easily countable?***	***Are objects starting to bled into its background?***	***Are objects almost same color as the rest of the image?***
Bright	Yes	No	No
Dim	Yes	Yes	No
Dark	No	No	Yes

The lumen is measured for each of the 10 photos for each classification and its luminance is averaged. The result from the following tests is

Bright = 107 lumen

Dim = 84 lumen

Dark = 37 lumen

The average lumen value of dim will be the separating value between criteria (a) and (b), meaning if a frame of a video feed or an image input has lumen value of higher than 84, model Mobilenet v2 FLNLite 640x640 trained on normal dataset will be used to detect objects under criteria (a). If the lumen value of the frame falls below 84, model Mobilenet v2 FPNLite 640x640 trained on grey scale version of the dataset will be used to detect objects which fall under criteria (b) which can include night time, raining, or cloudy environments.

V. RESULTS AND OUTOME

A. Result Analysis & Proposed Framework

The table and figure below list all the main components for the real world IoT implementation. The device is implemented in the Raspberry Pi as:

1. Its compact size is suitable for various applications with small-scale deployment
2. Its flexibility allows the device to be run on various operating systems such as a Linux/Debian-based distribution and Windows, offering users various software tools and libraries to utilize.
3. Its modularity allows easy access to connect with external deices and sensors.
4. Its active community provides extensive documentation, tutorials and support for any issues that arise with the device.

TABLE VIII. COMPONENTS FOR THE IOT IMPLEMENTATION

No	*Component*
1.	Raspberry Pi 4 Model B [8GB RAM]
2.	Logitech C270i HD Webcam
3.	Samsung EVO Plus MicroSD Card [64 GB]
4.	LCD Display

- Raspberry Pi 4 model B(Pi 4): is the chosen model as it offers the best performance of an IoT device compared to its other counterpart such as the Pi 3, Pi 3 Model B, and Pi 3 Model A+. The Pi 4 uses the (1) Broadcom BCM2711 processor, running approximately at 1.5 GHz clock speed, (2) equipped with the latest VideoCore's GPU, VideoCore VI, and (3) has up to 8 GB of RAM
- C279i HD Webcam: offers 720p resolution at 30 fps, with 55° field of view and an auto-light correction feature to preserve the quality of the photo.
- EVO Plus MicrSD Card: offers minimum transfer speed of class 10 (10 MB/s), up to 130 MB/s. Ensuring the transfer is fast and reliable to the connected IoT device.
- LCD Screen: offers a 5 inch LCD display with a resolution of 800x480 for viewing that fits within the small-form factor of the IoT device.

Fig. 17: Components for the IoT Implementation From left to right: Micro SD Card, C270i Webcam, LCD Screen, Pi 4 Model B

B. *Framework Performance*

The images chosen for criteria (a)(lumen > 84) and (b)(lumen < 84) are images from BDDA dataset test images. The following image name and lumen values are listed below for the purpose of replicability.

TABLE IX. PERFORMANCE TEST IMAGE INFORMATION

Image Information (Minimum 3 Vehicles Present)										
> 84 Lumens	0.jpg	3.jpg	6.jpg	12.jpg	15.jpg	18.jpg	19.jpg	22.jpg	25.jpg	47.jpg
Lumens	91.59	91.27	98.6	91.79	92.45	116.57	111.10	115.43	106.63	110.25
< 84 Lumens	2.jpg	4.jpg	7.jpg	10.jpg	14.jpg	9.jpg	23.jpg	32.jpg	64.jpg	109.jpg
Lumens	13.2	76.39	16.11	72.13	78.25	44.71	56.33	67.72	46.72	41.89

Before testing the performance of the framework, a benchmark needs to be established. We tested each model(model day; criteria (a) & model night; criteria (b) in isolation on all the images present in the table above. The test object detection number results in the 2 tables below correspond directly with the table above (Table 9). For example, the result for model:day lumen > 84 column 1 under "No of object detected" will correspond with that model's performance on image 0.jpg in the table above.

The threshold for each model is adjusted to 0.5 meaning it only takes into account if the confidence of that model's prediction is above 50%. This applies to both models to gauge its raw performance. The next test is dropping the threshold value to 0.3 to determine if there are characteristics that are useful for each model. For example if a model has high precision with low recall, dropping the threshold would theoretically improve its performance by a large margin. The night model (criteria (b)) that is trained on the grayscale images has these characteristics as shown in table 6.

1) *Test Terminology*

- i = Inconclusive, the bounding box are false positives or overlap

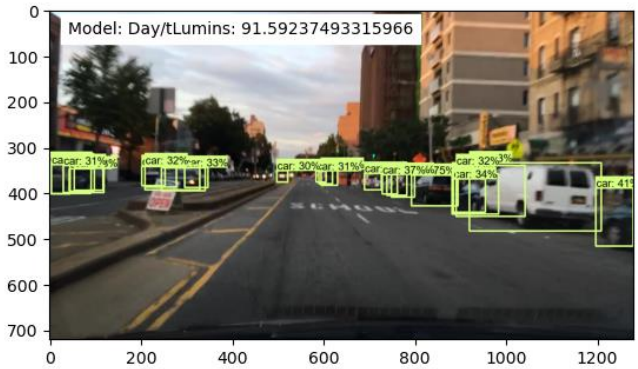


Fig. 18: Example of Inconclusive Test Output

- si = Slightly inconclusive, the bounding boxes for objects not directly in view or is far in distance from POV of camera are of (i)

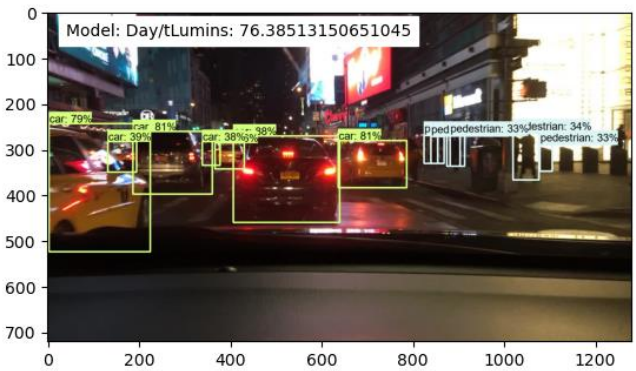


Fig. 19: Example of Slightly Inconclusive Test Output

TABLE X. PEROFRMANCE TEST RESULTS OF MODEL DAY (CRITERIA A)

Model: Day											
Img Lumen	No of Object Detected										Total
> 84	7	8	6	9	3	6	6	10	8	7	70
< 84	1	5	4	4	4	6	6	8	7	5	50
> 84 0.3	i	i	i	i	i	i	si	i	i	i	0
< 84 0.3	2	si	i	i	si	si	i	i	i	i	2

TABLE XI. PEROFRMANCE TEST RESULTS OF MODEL NIGHT (CRITERIA B)

Model: Night											
Img Lumen	No of Object Detected										Total
> 84	4	7	5	9	3	6	6	8	9	7	64
< 84	1	5	4	6	4	6	6	7	5	4	48
> 84 0.3	4	7	5	9	3	6	6	8	9	7	64
< 84 0.3	2	7	7	7	5	6	7	8	i	7	56

From the 2 tables, it can be surmised that Model: Day performs best when it is detecting objects for criteria (a)(lumen > 84) and Model: Night is best when detecting objects for criteria (b)(lumen < 84), however the threshold has to be reduced to 0.3 to uncover its full potential. Lowering the threshold for Model: Day seems to yield either inconclusive or slightly inconclusive detections, this is probably due to its high recall meaning if threshold is reduced overlapping detections will be made. The best option is to utilize both Model: Day with threshold of 0.5 and Model: Night with threshold of 0.3. Using table 9 and using the exact same models, the optimal results should be the exact same as the best for > 84 and < 84.

TABLE XII. PERFORMANCE TEST RESULTS OF FRAMEWORK

Model: Night											
Img Lumen	No of Object Detected										Total
> 84	7	8	6	9	3	6	6	10	8	7	70
< 84	2	7	7	7	5	6	7	8	i	7	56

As seen in the table above, the optimal results for > 84 and < 84 are replicated by the framework meaning if deployed in real use cases, it should outperform both previous models as well as other models explored in this paper.

Fig. 29: RaspberryPi Output Sample

VI. CONCLUSION & FUTURE WORKS

Obtaining complex road and traffic data is a difficult and tedious task to perform manually, our paper explored multiple object detection algorithms to determine which model is the most suited to mitigate and solve this data gathering issue. Strict requirements and testing had been conducted on the models to determine which model had the best performance in meeting the criteria (a) and (b). Mobilenet v2 FPV lite 640x640 had the best result amongst the four models, and implemented into an IoT device to determine the feasibility in a real-life setting. The result indicated that the device has very promising use cases in real-life situations.

There are some elements missing from this paper which could be explored in future contributions. One such example would be to test the viability of a single model which could perform well in both criteria (a) and (b), a potential option in order to achieve this could be to mix both normal training data and grayscale training data together and train the model from there, this avenue of research could either result in a more efficient framework or result in mediocre performance that generally performs well but not as well when matched against specialized models like the one in this paper. Whether this is the case is yet to be seen.

REFERENCES

[1] S. S. A. Zaidi, M. S. Ansari, A. Aslam, N. Kanwal, M. Asghar, and B. Lee, "A Survey of Modern Deep Learning based Object Detection Models," *arXiv.org*, Apr. 24, 2021. https://arxiv.org/abs/2104.11892

[2] M. Haris and A. Glowacz, "Road Object Detection: A Comparative Study of Deep Learning-Based Algorithms," *Electronics*, vol. 10, no. 16, p. 1932, Aug. 2021, doi: 10.3390/electronics10161932.

[3] G. Antipov, S.-A. Berrani, N. Ruchaud, and J.-L. Dugelay, "Learned vs. Hand-Crafted Features for Pedestrian Gender Recognition," in *Proceedings of the 23rd ACM international conference on Multimedia*, Oct. 2015. Accessed: Jan. 29, 2023. [Online]. Available: http://dx.doi.org/10.1145/2733373.2806332

[4] Z.-Q. Zhao, P. Zheng, S. Xu, and X. Wu, "Object Detection with Deep Learning: A Review," *arXiv.org*, Jul. 15, 2018. https://arxiv.org/abs/1807.05511

[5] H. M. Bui, M. Lech, E. Cheng, K. Neville, and I. S. Burnett, "Using grayscale images for object recognition with convolutional-recursive neural network," in *2016 IEEE Sixth International Conference on Communications and Electronics (ICCE)*, Jul. 2016. Accessed: Jan. 29, 2023. [Online]. Available: http://dx.doi.org/10.1109/cce.2016.7562656

[6] Y.-C. Chiu, C.-Y. Tsai, M.-D. Ruan, G.-Y. Shen, and T.-T. Lee, "Mobilenet-SSDv2: An Improved Object Detection Model for Embedded Systems," in *2020 International Conference on System Science and Engineering (ICSSE)*, Aug. 2020. Accessed: Jan. 29, 2023. [Online]. Available: http://dx.doi.org/10.1109/icsse50014.2020.9219319

[7] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and Efficient Object Detection," *arXiv.org*, Nov. 20, 2019. https://arxiv.org/abs/1911.09070

[8] J. Kim, J.-Y. Sung, and S. Park, "Comparison of Faster-RCNN, YOLO, and SSD for Real-Time Vehicle Type Recognition," in *2020 IEEE International Conference on Consumer Electronics - Asia (ICCE-Asia)*, Nov. 2020. Accessed: Jan. 29, 2023. [Online]. Available: http://dx.doi.org/10.1109/icce-asia49877.2020.9277040.

[9] Z. Chen, R. Khemmar, B. Decoux, A. Atahouet, and J.-Y. Ertaud, "Real Time Object Detection, Tracking, and Distance and Motion Estimation based on Deep Learning: Application to Smart Mobility," in *2019 Eighth International Conference on Emerging Security Technologies (EST)*, Jul. 2019. Accessed: Jan. 29, 2023. [Online]. Available: http://dx.doi.org/10.1109/est.2019.8806222

[10] M. Carranza-García, J. Torres-Mateo, P. Lara-Benítez, and J. García-Gutiérrez, "On the Performance of One-Stage and Two-Stage Object Detectors in Autonomous Vehicles Using Camera Data," *Remote Sensing*, vol. 13, no. 1, p. 89, Dec. 2020, doi: 10.3390/rs13010089.

[11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *arXiv.org*, Jan. 13, 2018. https://arxiv.org/abs/1801.04381

[12] W. Liu *et al.*, "SSD: Single Shot MultiBox Detector," *arXiv.org*, Dec. 08, 2015. https://arxiv.org/abs/1512.02325

[13] L. Yang, X. Zhang, L. Wang, M. Zhu, and J. Li, "Lite-FPN for Keypoint-based Monocular 3D Object Detection," *arXiv.org*, May 01, 2021. https://arxiv.org/abs/2105.00268

[14] A. G. Howard *et al.*, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv.org*, Apr. 17, 2017. https://arxiv.org/abs/1704.04861

[15] Y.-C. Chiu, C.-Y. Tsai, M.-D. Ruan, G.-Y. Shen, and T.-T. Lee, "Mobilenet-SSDv2: An Improved Object Detection Model for Embedded Systems," in *2020 International Conference on System Science and Engineering (ICSSE)*, Aug. 2020. Accessed: Jan. 29, 2023. [Online]. Available: http://dx.doi.org/10.1109/icsse50014.2020.9219319

[16] M. Cordts *et al.*, “The Cityscapes Dataset for Semantic Urban Scene Understanding,” *arXiv.org*, Apr. 06, 2016. https://arxiv.org/abs/1604.01685

[17] Y. Liao, J. Xie, and A. Geiger, “KITTI-360: A Novel Dataset and Benchmarks for Urban Scene Understanding in 2D and 3D,” *arXiv.org*, Sep. 28, 2021. https://arxiv.org/abs/2109.13410

[18] F. Yu *et al.*, “BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning,” *arXiv.org*, May 12, 2018. https://arxiv.org/abs/1805.04687

[19] tensorflow, “GitHub - tensorflow/models: Models and examples built with TensorFlow,” *GitHub*. https://github.com/tensorflow/models

[20] W. A. H. Rushton and H. D. Baker, “Red/green sensitivity in normal vision,” *Vision Research*, vol. 4, no. 1–2, pp. 75–85, May 1964, doi: 10.1016/0042-6989(64)90034-3.